

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart**
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115** Bug **15** Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name...

Widget constructors should have a key parameter
Code Smell
"SizedBox" should be used to add a whitespace to a layout
Code Smell
Flutter widget "createState" should only return a new "State"
Code Smell
@immutable classes should only have const constructors
Code Smell
Const constructors should be invoked with const
Code Smell
Web-only libraries should only be used in Flutter web plugins
Code Smell
Generic function type syntax should be preferred for parameters
Code Smell
Null-aware assignments should make sense
Code Smell
Unnecessary use of "toList" with spread operator
Code Smell
Unnecessary braces in string interpolation should be removed
Code Smell
Initializing formals should be used
Code Smell

Widget constructors should have a key parameter

Analyze your code

Intentionality - Complete Maintainability

Code Smell Major

Widget constructors should have a **key** parameter, forwarded to the superclass constructor.

- Why is this an issue?
- How can I fix it?
- More Info

Add a key parameter of type `Key` to each of the constructor of the widget, and forward it to the superclass constructor:

- either using a `super` parameter formal: `MyWidget({ this.key })`;
- or forwarding explicitly: `MyWidget({ Key? key }) : super(key: key)`;

Code examples

Noncompliant code example

```
class MyWidget extends Widget {
  MyWidget() : super(); // Non compliant
}
```

Compliant solution

```
class MyWidget extends Widget {
  MyWidget({ super.key });
}
```

Noncompliant code example

```
class MyWidget extends Widget {
  final int widgetConfig;

  MyWidget(this.widgetConfig) : super(); // Non compliant
}
```

Compliant solution

```
class MyWidget extends Widget {
  final int widgetConfig;

  MyWidget(this.widgetConfig, { Key? key }) : super(key: key);
}
```

Available In:
sonarcloud

